Carbon footprint of algorithms

Anuj Khadka Vladislav D. Veksler Darryl Aucoin

Abstract

This project investigates how programming language choice affects the carbon footprint produced during a program execution. Although software is often seen as intangible, every program consumes energy, ultimately contributing to carbon emissions. As the digital system continues to grow, understanding the environmental impact of software has become increasingly important. This experiment studies how identical algorithms behave when implemented in different programming languages running on the same hardware. By measuring and estimating the carbon emissions, the project tends to explore whether programming language designs play an important role in software sustainability, or if it is mostly driven by heavy algorithms. This research combines the aspects of computer science and environmental science to better understand the hidden environmental cost of everyday computational tasks and encourage more sustainable and responsible software development practices.
Keywords: carbon footprint, software sustainability, energy consumption

1 Introduction

As the tech continues to evolve, software becomes an increaingly central part of our lives. From the apps on our phones to the software running our cars, it's everywhere. But have you ever stopped to think about the environmental impact of all this software? Everytime a piece of code runs, it triggers energy usage inside the CPU, GPU, and other hardware components [**greenuk_footprint**]. This subsequent usage of software adds up to the carbon emission a device generates [**thoughtworks_carbon_intensity**]. The carbon footprint of software is a growing concern, as it contributes to climate change and has a significant impact on our planet.

The scale of this concern is larger than most people realize. The information and communication technology (ICT) sector accounts for an estimated of 2.1% to 3.9% of the global greenhouse gas emissions [**ict_stat**].

2 Background

In the following sections, we describe the energy consumption model, carbon intensity assumptions, programming languages, and the implementations of the six algorithms used in this study.

2.1 Energy Consumption

2.2 Carbon Intensity

2.3 Programming Language

The programming languages chosen for this study are C, Python, Javascript, Java, Go, and Rust. These languages were selected to represent a range of programming paradigms, performance characteristics, and popularity in the software development community. Each language has its own unique features and trade-offs that may influence the energy consumption and carbon footprint of the algorithms implemented in them [**decisionmodels**]. The performance characteristics of these languages are summarized in Table **??**.

Table 1: Programming languages, their performance characteristics, and popularity.

Language	Paradigm	Runtime	Version
C	Manual Memory, Compiled	GCC	14.3.0
Rust	Ownership, Compiled	Rustc	1.94.0
Python	Interpreted	CPython	3.12.10
Javascript	Dynamic, V8	Node.js	24.14.0
Java	OOP, JVM	OpenJDK	21.0.10
Go	GC	Go	1.26.1

2.3.1 Implementation-specific design decisions

Each language's implementation follows the stdin/stdout handshake protocol. The program

prints `"ready"` on startup, then blocks waiting for a newline trigger from the measurement harness before executing the algorithm. The specific I/O primitives used, and the rationale for each choice, are described below:

C - fgets

Python - stdin/stdout

JavaScript - readline

Java - public static void main
buffered reader/writer

Go - bufio

Rust - `wrapping_mul` 2654435761

Summation, Binary Search, Merge Sort, BFS, Hash Table, Matrix Multiplication why? because they represent different types of computational work, cpu-bound, memory-bound, branching-heavy etc.

2.4 Algorithms

The algorithms were chosen to span a range of time complexities and computational patterns, including CPU-bound, memory-bound, and branching-heavy workloads. The six algorithms used in this study are summarized in Table **??**.

Table 2: Algorithms, their time complexities and computational patterns.

Algorithm	Time Complexity	Pattern
Summation	$O(n)$	Linear
Binary Search	$O(\log n)$	Logarithmic
Merge Sort	$O(n \log n)$	Divide & Conquer
Breadth-First Search (BFS)	$O(V + E)$	Graph Traversal
Hash Table	$O(n) - avg$	xxxxxx
Matrix Multiplication	$O(n^3)$	Dense Arithmetic

Merge Sort - Now, just o make sure that we don't get any RecursionError in Python, we will increase the recursion limit for merge sort. Also,

Hash Table - the "& 0xFFFFFFFFFFFFFFFF" is used to ensure that the hash result is always a 64bit integer, which is important for consistency across different platforms and languages. This way, we can compare the hash results from C and Python without worrying about differences in integer sizes or overflow behavior.

Matrix Multiplication - Damn, this is slow. i should have used something more efficient. but maybe I will just leave this for now. I will change this if I really need to. but for now, I just want to go with the classic version.

o1 or on for hash table fix later. does insertions and lookups. each o1 on average, but worst case on if all keys collide.

3 Methodology

okay this is the intro

3.1 Experimental Setup

recording variance to prove the difference is real not noice.

Measurement Sequence - 01_verify output:

```
setup - not timed
print "ready" - not timed
harness newline - start timer
run command - measurement
print checksum - stop timer
```

fgets and sys.stdin and readline all blocks waiting for input until there is something to read from stdin/fgets. then reeads one line and returns it. So the harness will print "ready" and then wait for the next line. The next line is the signal to start the timer and run the command. After the command is done, the harness will print the checksum and stop the timer. This way we can measure the time taken by the command to run and also verify the output using checksum. For now, the command is "enter" btn, if I were to run the harness manually. But the verify file does this in the file itself.

Observation

```
- I can verify the thing now.
- The output that I get from the
```

```
baseline of C language can be
used to verify the time complexity
of the algorithms.
-
```

Table 3: C — RUN & VERIFY

Algorithm	Size	Output	Time	Status
summation	small	5050	0.0437ms	PASS
summation	mid	50005000	0.0406ms	PASS
summation	large	500000500000	0.4715ms	PASS
`binary_search`	small	99	0.0448ms	PASS
`binary_search`	mid	9999	0.0392ms	PASS
`binary_search`	large	999999	0.0835ms	PASS
`merge_sort`	small	100	0.0428ms	PASS
`merge_sort`	mid	10000	0.2459ms	PASS
`merge_sort`	large	1000000	24.7405ms	PASS
bfs	small	100	0.0430ms	PASS
bfs	mid	1000	0.3039ms	PASS
bfs	large	5000	7.2774ms	PASS
`hash_table`	small	64850	0.0433ms	PASS
`hash_table`	mid	649985000	0.2467ms	PASS
`hash_table`	large	64999850000	2.1579ms	PASS
`matrix_multiplication`	small	9175	0.0744ms	PASS
`matrix_multiplication`	mid	646700	2.5604ms	PASS
`matrix_multiplication`	large	10291750	47.1134ms	PASS

baseline power draw - 40w 5s sample
50reps each sample

4 Results

5 Discussion

6 Conclusion

7 Acknowledgement

This paper uses AI for few code writing processes, such as generating initial code snippets and providing suggestions for code improvements.

add some more later in this